

บทที่ 4

ASP.NET เว็บเซอร์วิส

เว็บเซอร์วิสช่วยให้เราเรียกใช้ ซอฟต์แวร์คอมพิวเตอร์ผ่านทางเว็บโปรโตคอลเช่น HTTP โดยใช้ อินเทอร์เน็ตและ XML เราสามารถสร้าง software component เพื่อติดต่อกับคนอื่นๆ โดยไม่ต้องสนใจในเรื่องของภาษา และแพลตฟอร์ม จากเดิมแต่ละค่ายจะมีการทำการติดต่อผ่านโปรโตคอลของตัวเอง มี อินเทอร์เน็ตที่แตกต่างกันไป ในเว็บเซอร์วิสสิ่งเหล่านี้จะเป็นมาตรฐานในการติดต่อส่งผ่านกันให้เป็นเรื่องง่าย โดยผ่าน SOAP และ XML เป็นเครื่องมือสำคัญ

4.1 องค์ประกอบในการใช้งาน .NET เว็บเซอร์วิส

4.1.1 รูปแบบการเชื่อมต่อ

Microsoft .NET Web Services ตอนนี้รองรับสามโปรโตคอล ได้แก่ HTTP GET, HTTP POST และ SOAP (Simple Object Access Protocol) เนื่องจากโปรโตคอลเหล่านี้เป็นมาตรฐานของเว็บอยู่แล้ว จึงง่ายที่ไคลเอนต์จะเรียกใช้

HTTP GET

query string จะรวมอยู่ใน URL ของเซิร์ฟเวอร์นั้น

HTTP POST

จะทำการรวม ชื่อ ค่าตัวแปรต่างๆ ไว้ในส่วนของบอดี ของ HTTP request message

SOAP

SOAP จะแตกต่างจากทั้งสองวิธีข้างต้น มันจะใช้ XML ในการจัดวางรูปแบบเมสเสจที่ส่งไปจะมีโครงสร้างที่ดีกว่าและสามารถส่งผ่านข้อมูลที่ซับซ้อนได้ ข้อแตกต่างอีกอย่างคือ SOAP สามารถใช้บน transport protocols อื่นๆได้ เช่น SMTP นอกเหนือจาก HTTP

4.1.2 WSDL

WSDL (Web Services Description Language) เกิดจากความร่วมมือระหว่าง IBM และ Microsoft WSDL เป็นภาษาที่ใช้อธิบายคุณลักษณะของ web service และวิธีการติดต่อกับ web service นั้น ๆ โดยใช้ ไวยากรณ์ของภาษา XML ซึ่ง WSDL อยู่ในความดูแลของ W3C version ล่าสุด คือ WSDL 1.1 ซึ่งหากผู้อ่านสนใจในรายละเอียด สามารถหาอ่านเพิ่มเติมได้จากเว็บไซต์ของ W3C ที่ <http://www.w3.org/TR/wsdl> ส่วนในทางปฏิบัติ หากเราต้องการ สร้าง web service ขึ้นมาเป็นของตนเอง ก็สามารถสร้างเอกสาร WSDL ได้โดยอัตโนมัติ เราจึงไม่ต้องไปกังวลในรายละเอียด ในข้อกำหนดใน WSDL มากนัก ซึ่ง

ทั้ง IDL และ WSDL นั้นอธิบาย อินเทอร์เฟซของ เมธอด ที่เรียกใช้งาน แล้ว แสดง พารามิเตอร์ที่ส่งเข้า ส่งออกด้วย สิ่งแตกต่างกันคือ คำอธิบายต่างๆใน WSDL จะอยู่ในรูปแบบของ XML

4.1.3 Web Services Discovery

เป็นกระบวนการในการค้นหาเซอร์วิสและทำการตรวจสอบ WSDL ของเว็บเซอร์วิสว่ามีอยู่จริงไหม แบ่งเป็น 2 แบบ

Static discovery

เราต้องทำการบอกชัดเจนในไฟล์ .disco ว่า wsdl อยู่ที่ไหน

```
<?xml version="1.0" ?>
```

```
<disco:discovery xmlns:disco="http://schemas.xmlsoap.org/disco">
```

```
</disco:discovery>
```

Dynamic discovery

เราสามารถตั้งค่าให้เป็นแบบ dynamic discovery ได้ ซึ่งจะทำให้เว็บเซอร์วิสทั้งหมดที่อยู่ภายใต้ URL ที่เรากำหนดจะถูกลิสต์ขึ้นมาโดยอัตโนมัติ

4.1.4 UDDI

UDDI (Universal Description, Discovery and Integration) เป็นมาตรฐานที่จัดตั้งขึ้น โดยบริษัท IBM, Microsoft และบริษัทยักษ์ใหญ่ทางธุรกิจ B2B (Business-to-Business) อื่น ๆ UDDI ถูกสร้างขึ้นเป็นมาตรฐาน ในการค้นหาบริการ web service สำหรับคู่ค้าทางธุรกิจ ซึ่งเปรียบได้กับฐานข้อมูล ขนาดใหญ่ ซึ่งมีข้อมูลของ web service ที่เปิดให้บริการ โดยที่ web site สำหรับค้นหา web service มีอยู่หลายที่อย่างเช่น

<http://uddi.microsoft.com/search.aspx>

<http://www-3.ibm.com/services/uddi/testregistry/find>

<http://uddi.org>

4.2 การสร้าง ASP.NET เว็บเซอร์วิส

4.2.1 ขั้นตอนในการสร้าง Web Service

1. สร้างไฟล์ .asmx สำหรับ web service ที่สร้างด้วย ASP.NET ต้องประกอบไปด้วย directive

```
<% webservice ... %>
```

 การเขียนก็คล้ายกับการเขียนคลาส
2. WebService Class นั้นต้องทำการ inherit มาจาก System.Web.Services namespace รวมทั้งทำการกำหนด namespace ของเว็บเซอร์วิสของคุณด้วยเนื่องจาก default ของมันคือ <http://tempuri.org/> ซึ่งจะไม่ unique

3. เมธอดที่ต้องการให้เป็น เมธอดสาธารณะที่คนอื่น ๆ เข้ามาเรียกใช้ได้ ต้องทำการกำหนดให้เป็น
<Web Method> ด้วย
4. นำไฟล์ .asmx ที่สร้างไปไว้ในไดเรกทอรีที่ต้องการใน IIS

4.2.2 ตัวอย่างเว็บเซอร์วิส

```
<%@ WebService Language="VB" Class="Math"%>
```

```
Imports System.Web.Services
```

```
Imports System
```

```
<WebService(Namespace:="http://www.olala.com/")> _
```

```
Public Class Math
```

```
<WebMethod()> _
```

```
Public Function Add(num1 As Integer, num2 As Integer) _
```

```
As Integer
```

```
Return num1 + num2
```

```
End Function
```

```
<WebMethod()> _
```

```
Public Function Hello() _
```

```
As String
```

```
Return "Hello World"
```

```
End Function
```

```
End Class
```

4.2.3 การกำหนดค่าแอททริบิวต์ต่างๆให้แก่เว็บเซอร์วิส

BufferResponse (boolean)

ควบคุมให้มี buffer ของผลตอบรับของเมธอดหรือไม่

CacheDuration

กำหนดค่าช่วงเวลา(วินาที) ที่ทำการเก็บ method response ใน cache ถ้าปกติคือ 0 วินาที

Description

กำหนดคำอธิบายของเมธอดนี้

EnableSession (boolean)

ให้มีการใช้ เซสชันสำหรับเมธอดนี้หรือไม่ โดยค่าปกติเป็น true ถ้าเราไม่ได้ ใช้งานควรกำหนดเป็น false เพื่อเพิ่มประสิทธิภาพ

MessageName

เพื่อใช้ในการแย่งแยกเว็บเมธอดในกรณีที่มี ชื่อของเมธอดเหมือนกัน

TransactionOption

เป็นการกำหนด transaction ของเมธอดนั้น มีทั้งหมด 5 แบบ ดังที่ได้กล่าวไปแล้วในเรื่อง Transaction ได้แก่

1. Disabled - object ที่สร้างขึ้นจะไม่มี transaction ใน COM+ transaction แต่สามารถติดต่อโดยตรงกับ DTC ในกรณีที่ต้องการการสนับสนุน
2. NotSupported - ไม่มีการทำ transaction เลย
3. Supported - ถ้า object นั้นจะถูกรันใน context transaction ของตัวที่สร้างมันขึ้นซึ่งมีการทำ transaction , แต่ถ้าตัวมันเอง เป็นตัวเริ่ม(root - ไม่ได้ถูกใครสร้างมาอีกที) หรือ context ที่สร้างมัน ไม่ทำ transaction มันก็จะไม่ทำ transaction
4. Required - ถ้า object นั้นจะถูกรันใน context transaction ของตัวที่สร้างมันขึ้นซึ่งมีการทำ transaction , แต่ถ้าตัวมันเอง เป็นตัวเริ่ม(root - ไม่ได้ถูกใครสร้างมาอีกที) หรือ context ที่สร้างมัน ไม่ทำ transaction มันก็จะทำ transaction ใหม่
5. RequiresNew - บอกว่า object ต้องทำ transaction ใหม่

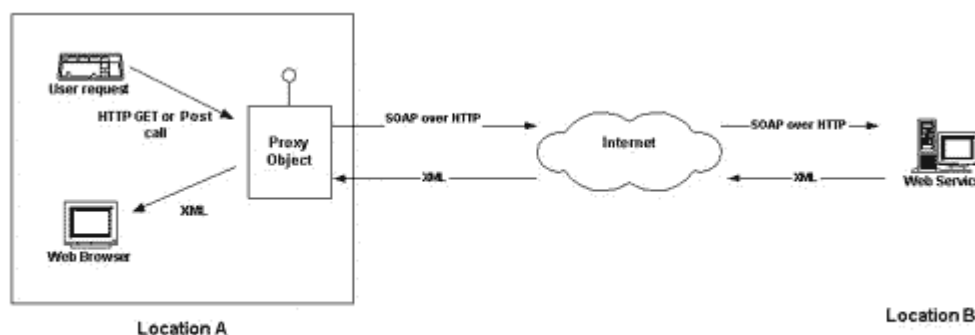
4.3 การเรียกใช้งานเว็บเซอร์วิส

ในการเรียกใช้งานเว็บเซอร์วิสนั้นเราสามารถที่จะเรียกใช้ได้ทั้ง 3 แบบ คือ HTTP GET, HTTP POST , SOAP

HTTP GET นั้นเราสามารถเรียกใช้ Web Service ได้โดยตรงผ่านทาง URL โดยใส่ parameter รวมไปด้วยเลข เช่น

<http://localhost/Xtest.asmx/Add?num1=3&num2=6>

แต่ในส่วนของ HTTP POST และ SOAP นั้นเราต้องทำการสร้าง Proxy Class ขึ้นมาก่อนเพื่อทำการติดต่อกับ Web Service



รูปที่ 4-1 แสดงการติดต่อจากไคลเอนต์ไปยังเว็บเซิร์ฟเวอร์ผ่านพร็อกซีอ็อบเจกต์

ขั้นตอนการสร้าง Proxy Class

1. ใช้ wsdl.exe ในการคอมไพล์โดยใช้พารามิเตอร์เป็น ชนิดโปรโตคอล, ตำแหน่งของเอกสาร wsdl ,ภาษาที่ใช้ และอื่นๆ
2. จะได้ไฟล์ออกมา .cs ซึ่งเป็นไฟล์ของ C# เราใช้ตัวคอมไพล์ของ C# ก็จะได้ไฟล์ .dll ออกมา
3. นำไฟล์ .dll นี้ไป add reference ในโปรเจกต์ที่จะเรียกใช้

4.4 การจัดการเซตในเว็บเซอร์วิส

ในส่วนของ ASP.NET มีฟีเจอร์ในการจัดการเซต ทั้งในส่วนในระดับแอปพลิเคชันและระดับของเซสชันของผู้ใช้ ในส่วนนี้เราจะกล่าวถึงการใช้งานเซตของ ASP.NET เพื่อนำมาพัฒนาเซตฟูลเว็บเซอร์วิส

คำว่า เซต นั้น เราอาจจะนิยามได้ว่า มันคือความสามารถของอ็อบเจกต์,ตัวแปร หรือคอนเทนเนอร์อื่นๆที่สามารถจดจำค่าของมันได้ ซึ่งเรามีอยู่สองระดับคือ แอปพลิเคชัน และเซสชัน ในระดับแอปพลิเคชันนั้น เปรียบเหมือนร้านอาหาร บริกรต้องจำได้ว่า วันนี้ร้านมีเมนูพิเศษอะไรบ้าง ซึ่งจะนำไปบอกลูกค้าทุกโต๊ะเหมือนกันหมด แต่ระดับเซสชัน บริกรจะจำได้ว่า ลูกค้าคนนี้สั่งอะไร แต่ละคนสั่งคนละอย่างไม่เหมือนกัน

ทั้งสองระดับที่กล่าวมานั้นใน ASP.NET เรียกว่า แอปพลิเคชันอ็อบเจกต์ ซึ่งให้บริการ กระบวนการในการจัดเก็บข้อมูล ซึ่งสามารถเข้าถึงได้จากทุกโค้ด ที่อยู่ในเว็บแอปพลิเคชันนั้น และ เซสชันอ็อบเจกต์ ซึ่งจัดเก็บข้อมูลตามเซสชันของไคลเอนต์แต่ละคนที่เข้ามา

4.4.1 แอปพลิเคชันอ็อบเจกต์

แอปพลิเคชัน อ็อบเจกต์จะถูกสร้างขึ้น หลังจากเกิดการเรียกใช้เว็บหรือเว็บเซอร์วิสนั้นขึ้นเป็นครั้งแรกจนกระทั่ง ปิดระบบเว็บเซอร์วิสลงไป ซึ่งแอปพลิเคชันอ็อบเจกต์นั้นให้การข้อดีต่างๆดังนี้

- ง่ายต่อการเรียกใช้เซต และสามารถได้จากภาษาใดๆก็ได้ที่ .NET รองรับ
- มีฟังก์ชันในการจัดการ ซึ่งใครในเซสชัน ทำให้นักพัฒนาง่ายต่อการจัดการการเข้าถึง ตัวแปรแบบโอบอลใน แอปพลิเคชันเซต
- แอปพลิเคชันเซต เข้าถึงได้เพียงโค้ดในส่วนของแอปพลิเคชันนั้นๆเท่านั้น ไม่สามารถที่แอปพลิเคชันอื่นจะเข้าถึงได้

ตัวอย่างการเรียกใช้งาน แอปพลิเคชัน อ็อบเจกต์

```
Application["DatabaseName"] = "DataStore" ;
```

4.4.2 เซสชันอ็อบเจกต์

เซสชันหมายถึง ช่วงเวลาที่ผู้ใช้คนๆหนึ่งทำการติดต่อกับเว็บแอปพลิเคชันหนึ่ง ซึ่งใน ASP.NET มีการทำงานที่ดีขึ้นกว่า ASP เวอร์ชันก่อนหน้าดังนี้

- เซสชัน อ็อบเจกต์โปรเซสจะถูกแยกออกมา ไม่ได้อยู่ในโปรเซสของ ASP เหมือนของเดิม ดังนั้นถ้าโปรเซสของ ASP ล้มเหลว ตัวเซสชันก็ยังอยู่

- เซสชัน อ็อบเจกต์ใน ASP.NET ให้การสนับสนุนการใช้ เซิร์ฟเวอร์หลายๆตัว(server farm) จากแนวคิดของ out-of-process ข้างต้น เราจึงสามารถทำให้ เซิร์ฟเวอร์หลายๆใช้ เซสชัน เสถพร้อมกันได้
- เซสชันอ็อบเจกต์ สามารถเรียกใช้โดยไม่ผ่าน คุกกี้ ได้ง่าย

การจัดการของเซสชันนั้น จะบอกได้จาก เลขเซสชันไอดี ซึ่งเป็นเลข 128 บิต หรือเรารู้จักกันดีว่า GUID's(Globally Unique Identifier) ซึ่งถูกสร้างขึ้นจากอัลกอริทึมที่มั่นใจว่าจะไม่เกิดการซ้ำกัน เลขเซสชันนี้จะถูกจัดเก็บในสองรูปแบบ คือ

1. ผ่านคุกกี้ ซึ่งคุกกี้ นั้นสามารถบ่งบอกถึงเซสชันของไคลเอนต์

2. จากการรวมค่าของไอดี เข้าไปยัง URL โดยตรง

เว็บเซอร์วิสนั้นก็ต้องการตัวในการจัดการเสถพบกับไคลเอนต์ เครื่องไคลเอนต์ที่จะเรียกใช้เว็บเซอร์วิสต้องสนับสนุนการใช้งานคุกกี้ด้วย เพื่อที่จะเก็บ เซสชันไอดี

ตัวอย่างการใช้งาน เซสชันอ็อบเจกต์ใน ASP.NET

```
Session["UserName"] = "John Smith" ;
```

4.4.3 โหมดการจัดการเซสชันและการคอนฟิกค่า

การคอนฟิกค่าต่างๆใน ASP.NET แบ่งออกได้เป็นสองส่วนด้วยกัน คือ machine.config และ web.config ไฟล์แรกนั้นจะส่งผลกับทุกเว็บแอปพลิเคชันในเครื่อง และ ไฟล์หลังจะเป็นการคอนฟิกเฉพาะแอปพลิเคชันซึ่งจะทับแมชชีนคอนฟิกอีกที

โหมดในการใช้งานเซสชันนั้นมีด้วยกัน 3 โหมด ได้แก่

1. In Proc เป็นค่าเริ่มต้นที่ติดตั้ง คือจะจัดเก็บเซสชันในโปรเซสเดียวกับเว็บเซิร์ฟเวอร์
2. State Server ทำการจัดเก็บลงในแบบ out-of-process ลงในเมมโมรี่ โดยเครื่องที่จะทำเซิร์ฟเวอร์นี้สามารถเป็นเครื่องเดียวกับ เว็บเซิร์ฟเวอร์ได้

```
stateConnectionString="tcpip=127.0.0.1:42424"
```

3. SqlServer ทำการจัดเก็บลงในแบบ out-of-process เช่นกัน แต่จัดเก็บลงในดาต้าเบส

```
sqlConnectionString="data source=127.0.0.1;user id=sa;password="
```

ค่าอื่นๆที่สามารถปรับแต่งได้ก็คือ โหมดของการใช้ คุกกี้ ว่าจะเป็นแบบ ใช้คุกกี้ หรือว่า ผ่าน url และการกำหนดค่าช่วงเวลาของคุกกี้

```
cookieless="true"
```

```
timeout="40"
```

ในเรื่องของประสิทธิภาพ โหมด In Proc มีประสิทธิภาพมากกว่า State Server และ SqlServer ตามลำดับ แต่ สองโหมดหลังจะสามารถใช้กับแบบ เซิร์ฟเวอร์ฟาร์มได้ โหมด SqlServer ก็จะช้าที่สุดแต่ก็มีความน่าเชื่อถือที่สุดเช่นกัน

4.4.4 การเรียกใช้งาน แอปพลิเคชัน และ เซสชันสเตทใน เว็บเซอร์วิส

เว็บเซอร์วิสเป็นส่วนหนึ่งของ ASP.NET จึงสามารถใช้งานในส่วนสเตทนี้ทำให้เป็น สเตทฟูลเว็บเซอร์วิสได้ ซึ่งการใช้งานนั้นต้องทำการเซตค่าของเว็บเมธอดเพื่อให้จำค่าเซสชันได้

```
[WebMethod(EnableSession:=True)]
```

ในกรณีที่เมธอดนั้นมีการใช้งาน แอปพลิเคชันอ็อบเจกต์เท่านั้น ไม่จำเป็นต้องกำหนดค่าดังกล่าว

การเขียนแอปพลิเคชัน เรียกใช้งานเว็บเซอร์วิส

การเรียกใช้งานนั้นอย่างทีบอกไปแล้ว ว่าต้องใช้คูกี้เพื่อให้จำเลขเซสชัน ไปได้

1. ทำการสร้างเว็บเซอร์วิสอ็อบเจกต์ขึ้นมา

```
localhost.StatefulService objService = new localhost.StatefulService();
```

2. สร้าง CookieContainer ซึ่งต้อง using System.Net ด้วย

```
CookieContainer objCookie = new CookieContainer();
```

3. ทำการกำหนดค่าให้แก่ เว็บเซอร์วิสอ็อบเจกต์

```
objService.CookieContainer = objCookie;
```

ถ้าเป็นในส่วนของเว็บฟอร์มนั้น ต้องมีการกำหนดค่า

```
if(Session["Container"] == null)
```

```
{
```

```
    objContainer = new CookieContainer(); // first call
```

```
    Session["Container"] = objContainer;
```

```
}
```

```
else
```

```
    objContainer = (CookieContainer)Session["Container"]; // all subsequent
```

```
callwb.CookieContainer = objContainer;
```

เพื่อให้ทุกครั้งที่มีการโหลดเพจใหม่ขึ้นมา ยังจำค่าคูกี้ได้

4.5 ทรานแซกชัน

4.5.1 เหตุผลที่ต้องใช้ทรานแซกชัน

ตัวอย่างปัญหาที่ต้องใช้ทรานแซกชันในการแก้

■ Atomic Operation

กรณีที่ต้องการทำคำสั่งหลาย ๆ คำสั่ง โดยมองทั้งหมดเป็นเหมือนคำสั่งเดียว (atomic operation) เช่น กรณีบัญชีธนาคาร เมื่อต้องการโอนเงินจากบัญชีหนึ่งไปยังอีกบัญชีหนึ่ง ต้องถอนเงินออกจากบัญชีหนึ่ง จากนั้นจึงฝากเงินเข้าอีกบัญชีหนึ่ง ถ้ามีคำสั่งใดคำสั่งหนึ่งล้มเหลว ต้องให้คำสั่งทั้งคู่ล้มเหลว ไม่เช่นนั้นยอดเงินคงเหลือในบัญชีจะไม่ถูกต้อง จึงต้องไม่เกิดสถานการณ์ที่คำสั่งหนึ่งทำงานสำเร็จ ในขณะที่อีกคำสั่งทำงานล้มเหลว เนื่องจากทั้งสองคำสั่งนั้นเป็นทรานแซกชันเดียวกัน

การแก้ปัญหานี้วิธีหนึ่งคือการใช้ exception ดังตัวอย่างต่อไปนี้

```
try {
    // Withdraw funds from account 1
}
catch (Exception e) {
    // If an error occurred, do not proceed.
    return;
}
try {
    // Otherwise, deposit funds into account 2
}
catch (Exception e) {
    // If an error occurred, do not proceed,
    // and redeposit the funds back into account 1.
    return;
}
```

ตัวอย่างโค้ดข้างต้น ถอนเงินจากบัญชีที่ 1 ก่อน หากเกิดข้อผิดพลาดขึ้น แอปพลิเคชันจะจบการทำงาน และไม่มีผลของการทำงานใด ๆ เกิดขึ้น หากไม่เกิดข้อผิดพลาดก็จะฝากเงินเข้าไปยังบัญชีที่ 2 ในขณะฝากเงินนั้น หากเกิดข้อผิดพลาด จะต้องทำการฝากเงินกลับเข้าไปในบัญชีที่ 1 ใหม่ จากนั้นจึงจบการทำงาน

ปัญหาที่เกิดขึ้นจากการแก้ปัญหาด้วยวิธีนี้คือ

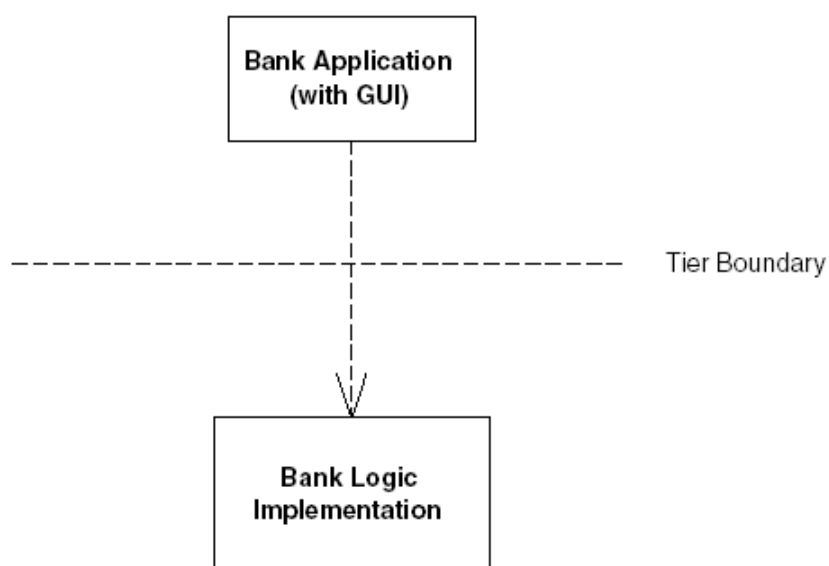
- โปรแกรมมีความซับซ้อน
- ต้องตรวจสอบทุกความเป็นไปได้ที่จะทำให้เกิดการทำงานผิดพลาดในทุก ๆ ขั้นตอน และต้องเขียนโค้ดเพื่อยกเลิกการเปลี่ยนแปลงที่ได้ทำไปก่อนที่จะเกิดความผิดพลาดนั่นเอง
- ในการตรวจสอบความผิดพลาด แทบจะไม่สามารถควบคุมได้หากกระบวนการทั้งหมดมีความซับซ้อน เช่นการปรับปรุงข้อมูลทางการเงิน ที่ต้องอาศัยการทำงานทั้งหมด 10 ขั้นตอนด้วยกัน จะต้องหาวิธีการในการรองรับการทำงานที่ผิดพลาดในทุกขั้นตอนที่เป็นไปได้
- การตรวจสอบการทำงานได้ยาก เพราะต้องทำการจำลองการทำงานที่ผิดพลาดที่เกิดขึ้นได้ในทุกขั้นตอน

ในทางอุดมคตินั้นต้องการวิธีการที่สามารถทำคำสั่งทั้งสองได้ โดยมองเป็นคำสั่งใหญ่เพียงคำสั่งเดียว และรับประกันว่าคำสั่งทั้งสองจะต้องทำงานสำเร็จทั้งคู่ หรือทำงานผิดพลาดทั้งคู่เท่านั้น

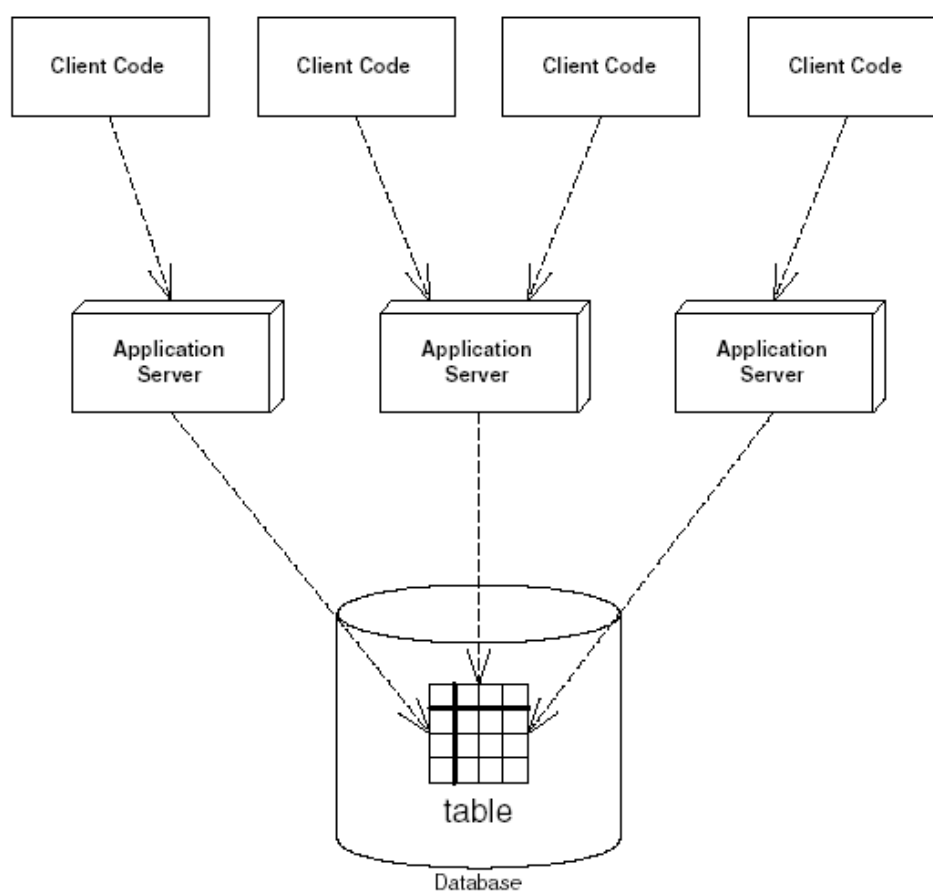
■ การทำงานผิดพลาดของเครื่องคอมพิวเตอร์ หรือระบบเครือข่าย

จากตัวอย่างข้างต้น ถ้าลอจิกต่าง ๆ ที่ทำงานกับบัญชีธนาคารถูกกระจายบนเครือข่ายโดยดีพลอยแบบมัลติเทียร์ เพื่อเหตุผลทางด้านความปลอดภัย , การเพิ่มขยาย หรือประโยชน์จากการแบ่งโปรแกรมออกเป็นโมดูล ในการดีพลอยแบบมัลติเทียร์ โคลเอนต์ที่ต้องการใช้งานแอปพลิเคชันบัญชีธนาคารนี้ จะต้องกระทำผ่านทาง remote method invocation แสดงในรูปที่ 4-2 ทำให้ต้องคำนึงถึงเรื่องของการทำงานผิดพลาดและความน่าเชื่อถือ เช่น ระบบเครือข่ายเครือข่ายระหว่างที่โปรแกรมบัญชีธนาคารกำลังทำคำสั่งใด ๆ อยู่ แม้ว่าเมื่อระบบเครือข่ายเครือข่ายจะมีการสร้าง exception ขึ้นมา แต่การแครชของระบบเครือข่ายนั้นอาจจะเกิดขึ้นก่อนหน้าการถอนเงิน หรืออาจเกิดขึ้นภายหลังการถอนเงินก็ได้ ซึ่งไม่อาจแยกความแตกต่างระหว่าง exception ที่เกิดขึ้นจากกรณีทั้งสองนี้ได้เลย

ในความเป็นจริง ระบบเครือข่ายไม่ได้เป็นจุดเดียวที่ทำให้เกิดปัญหานี้ขึ้น เนื่องจากข้อมูลบัญชีธนาคารก็คือข้อมูลแบบ persistent ที่อยู่ในฐานข้อมูล มีความเป็นไปได้เสมอที่ระบบฐานข้อมูลจะแครช หรือเครื่องคอมพิวเตอร์ที่ดีพลอยระบบฐานข้อมูลไว้จะเกิดแครช ถ้าหากการแครชนั้นเกิดขึ้นในระหว่างที่กำลังเขียนข้อมูลลงไปในฐานข้อมูล อาจทำให้ฐานข้อมูลอยู่ในสถานะที่ไม่ consistent ได้



รูปที่ 4-2 แอปพลิเคชันที่ทำงานในระบบแบบกระจาย



รูปที่ 4-3 การใช้งานฐานข้อมูลพร้อม ๆ กัน

■ การใช้ผู้หลายคนสามารถใช้งานข้อมูลเดียวกันพร้อมกันได้

ในระบบแบบ enterprise-level distributed system เป็นรูปแบบที่ไคลเอนต์หลาย ๆ ตัวติดต่อกับแอปพลิเคชันเซิร์ฟเวอร์หลาย ๆ ตัว ถ้าแอปพลิเคชันเซิร์ฟเวอร์ทั้งหมดใช้ฐานข้อมูลเดียวกัน (ดังรูปที่ 4-3) เซิร์ฟเวอร์สามารถที่จะแก้ไขข้อมูลตัวเดียวกันที่อยู่ในฐานข้อมูลได้

ถ้ามีไคลเอนต์สองตัวพยายามแก้ไขข้อมูลเดียวกันที่อยู่ในฐานข้อมูลในเวลาเดียวกันแล้ว คำสั่งของไคลเอนต์ทั้งสองอาจทำงานซ้อนกัน และได้ผลลัพธ์สุดท้ายผิดพลาด ซึ่งการมีข้อมูลที่ไม่ถูกต้องในฐานข้อมูล ในระบบทั่วไปถือว่าเป็นสิ่งที่ยอมรับไม่ได้ (unacceptable) และอาจสร้างความเสียหายอย่างมากให้กับองค์กรดังนั้นจึงจำเป็นต้องหาวิธีการที่จะจัดการกับการแก้ไขข้อมูลพร้อมกันของไคลเอนต์หลาย ๆ ตัว โดยจะต้องไม่ทำให้ผลลัพธ์สุดท้ายที่ได้นั้นเป็นผลลัพธ์ที่ไม่ถูกต้อง

4.5.2 ประโยชน์ของทรานแซกชัน

ทรานแซกชัน คือเซตของคำสั่งที่ถูกเอ็กซิกิวต์โดยมองว่าเป็นคำสั่งใหญ่เพียงอันเดียว ทรานแซกชันรับประกันว่าคำสั่งทั้งหมดทำงานสำเร็จ มิฉะนั้นจะต้องไม่มีผลการทำงานจากคำสั่งใดเกิดขึ้นเลย ทรานแซกชันสามารถแก้ปัญหาที่เกิดจากการทำงานผิดพลาดของระบบเครือข่ายได้ อนุญาตให้ไคลเอนต์หลาย ๆ ตัวใช้งานข้อมูลเดียวกันพร้อมกันได้ และรับประกันว่าข้อมูลทั้งหมดที่คำสั่งเหล่านี้ทำการแก้ไขจะถูกต้องสมบูรณ์ โดยไม่ถูกรบกวนจากการแก้ไขข้อมูลโดยไคลเอนต์อื่น ๆ

การใช้ทรานแซกชันอย่างถูกต้องทำให้สามารถมีหลายไคลเอนต์ทำงานกับฐานข้อมูลได้โดยไม่ขึ้นต่อกัน เช่น ถ้ามีไคลเอนต์สองตัวอ่านและเขียนข้อมูลลงในฐานข้อมูล ไคลเอนต์ทั้งสองจะ mutual exclusive กัน ระบบจัดการฐานข้อมูลจะจัดการเกี่ยวกับการควบคุมการทำงานพร้อมกันที่จำเป็นบนฐานข้อมูล เช่น การล็อกข้อมูล เพื่อไม่ให้งานของแต่ละเซรมีผลกระทบซึ่งกันและกัน

4.5.2.1 ACID Properties

เมื่อใช้งานทรานแซกชันอย่างถูกต้อง คำสั่งจะถูกเอ็กซิกิวต์ โดยมีการรับประกันผลการทำงานทั้งสี่ 4 อย่างด้วยกัน เรียกว่า ACID properties โดยคำว่า ACID ย่อมาจาก Atomicity , Consistency , Isolation และ Durability

- **Atomicity** - รับประกันว่าคำสั่งทั้งหมดจะถูกรวมเข้าไว้ด้วยกัน และมองเป็นหน่วยของการทำงานเพียงอันเดียว การทำงานของทรานแซกชันจะเป็นไปในลักษณะของ All-or-Nothing คือการอัปเดตข้อมูลในฐานข้อมูลทั้งหมดจะต้องเสร็จสิ้น หรือถ้าเกิดข้อผิดพลาดใด ๆ ขึ้นระหว่างการดำเนินงาน การอัปเดตฐานข้อมูลทั้งหมดนั้นจะต้องไม่เกิดขึ้นเลย โดยการอัปเดตฐานข้อมูลที่เกิดขึ้นก่อนแล้ว จะต้องถูกยกเลิก
- **Consistency** - รับประกันว่าทรานแซกชันจะต้องทำให้ระบบอยู่ในสถานะที่ consistent เมื่อทรานแซกชันทำงานเสร็จเรียบร้อย เช่น การโอนเงินจะต้องไม่มีเงินหายไปจากระบบ แต่ใน

ระหว่างการทำงานของทรานแซกชันนี้ ขณะที่เงินถูกถอนออกมาจากบัญชีแรก ข้อมูลอยู่ในสถานะที่ไม่ consistent แต่เนื่องจากทรานแซกชันจะมองเห็นคำสั่งทั้งหมดเปรียบเสมือนเป็นคำสั่งเดียว ดังนั้นถ้าทรานแซกชันสำเร็จก็คือคำสั่งทั้งหมดจะต้องสำเร็จ แต่ถ้าทรานแซกชันทำงานไม่สำเร็จคำสั่งทั้งหมดก็ต้องไม่สำเร็จ และการเปลี่ยนแปลงใด ๆ ที่ได้กระทำไประหว่างนั้นจะต้องถูกยกเลิก ตามคุณสมบัติข้อแรกของทรานแซกชัน ดังนั้นข้อมูลจะอยู่ในสถานะที่ consistent เสมอ

- **Isolation** - เป็นการป้องกันทรานแซกชันที่ทำงานพร้อมกัน ไม่ให้เห็นผลลัพธ์ที่ยังไม่สมบูรณ์ที่เกิดจากทรานแซกชันอื่น isolation อนุญาตให้หลายทรานแซกชันสามารถอ่านหรือเขียนฐานข้อมูลโดยไม่รู้ถึงการมีอยู่ของทรานแซกชันอื่น ถ้าไม่มีคุณสมบัติ isolation สถานะของข้อมูลอาจอยู่ในสถานะที่ไม่ consistent ได้ ซึ่งมีประโยชน์มากในกรณีไคลเอนต์หลายตัวแก้ไขฐานข้อมูลในเวลาเดียวกัน โดยแต่ละไคลเอนต์จะทำงานโดยเปรียบเสมือนว่ามีเพียงตัวมันเองเท่านั้นที่กำลังใช้งานฐานข้อมูลอยู่ ทรานแซกชันสร้างคุณสมบัติ isolation โดยการใช้ synchronization โปรโตคอลต่าง ๆ กับข้อมูลในฐานข้อมูล เช่น ระหว่างการทำงานของทรานแซกชัน ข้อมูลจะถูกล็อกและปลดล็อกโดยอัตโนมัติตามความจำเป็น และการอัปเดตข้อมูลของทรานแซกชันอื่น จะไม่มีผลกับข้อมูลที่ทรานแซกชันนี้กำลังทำงานอยู่
- **Durability** - รับประกันว่าการเปลี่ยนแปลงข้อมูลใด ๆ ในฐานข้อมูลที่เสร็จสิ้น (commit) แล้วจะต้องมีความถาวร คือต้องไม่สูญหายเมื่อเกิดความผิดพลาดใด ๆ ขึ้น เช่น เครื่องคอมพิวเตอร์แครช, ระบบเครือข่ายแครช, ฮาร์ดดิสก์แครช หรือไฟฟ้าดับ โดยต้องมีการเก็บ log file ของการอัปเดตข้อมูลใด ๆ ในทรานแซกชัน เมื่อระบบเกิดแครชขึ้น ข้อมูลสามารถสร้างขึ้นมาใหม่โดยการทำงานตามลำดับใน log file อีกครั้งหนึ่ง

4.5.3 ทรานแซกชันโมเดล

ทรานแซกชันโมเดลเป็นวิธีการที่ต่างกันในการทำทรานแซกชัน มีหลายโมเดลที่ใช้จัดการทรานแซกชัน แต่ละโมเดลจะมีความซับซ้อนและฟีเจอร์ที่ต่างกัน 2 โมเดลที่ได้รับความนิยมมากที่สุดคือ flat transaction และ nested transaction

- **Flat Transactions**

flat transaction เป็นทรานแซกชันโมเดลที่เข้าใจได้ง่าย flat transaction เป็นกลุ่มของ operation ที่ทำงานเป็นหน่วยเดียว ถ้า operation ไม่สำเร็จทั้งหมดจะเกิด rollback ขึ้น แต่ถ้าสำเร็จจะเกิด commit กระบวนการของ flat transaction ดังรูปที่ 4-4

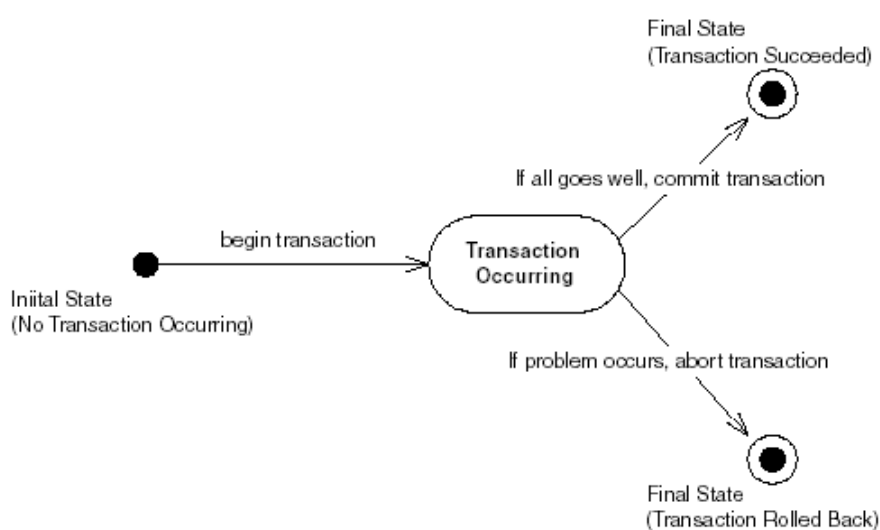
- **Nested Transactions**

ตัวอย่างของงานที่ต้องใช้ nested transaction คือปัญหา trip-planning โดยแอปพลิเคชันจะมี operation ดังนี้

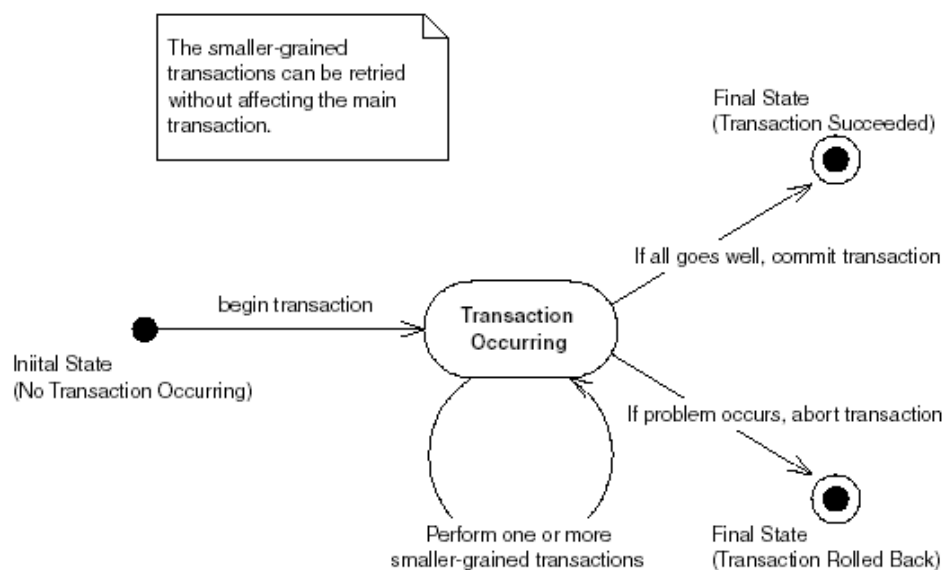
1. ซื้อตั๋วรถไฟจากบอสตันไปยังนิวยอร์ก
2. ซื้อตั๋วเครื่องบินจากนิวยอร์กไปยังลอนดอน
3. ซื้อตั๋วบอลลูนจากลอนดอนไปยังปารีส
4. แอปพลิเคชันพบว่าไม่มีไฟล์ที่ไปยังปารีส

ถ้าลำดับของ operation นี้ทำกับ flat transaction จะเกิด rollback เท่านั้น เนื่องจากไม่มีไฟล์ที่ไปยังปารีส แอปพลิเคชันจะต้องไม่เก็บการจองใด ๆ ไว้เลย แต่ตรงจุดนี้อาจมีทางในการเปลี่ยนการเดินทางด้วยบอลลูนไปเป็นทางอื่น ทำให้เก็บข้อมูลการจองไว้ได้

nested transaction ช่วยในการแก้ปัญหานี้ มันจะยอมให้เกิดทรานแซกชันซ้อนกันได้ รูปที่ 4-5 แสดงแนวคิดของ nested transaction



รูปที่ 4-4 Flat Transaction



รูปที่ 4-5 Nested Transaction

4.5.4 Transactional Isolation

คุณสมบัติ isolation รับประกันว่า ผู้ใช้ที่ทำงานอยู่พร้อม ๆ กันจะไม่รู้สึกและได้รับผลกระทบจากผู้ใช้คนอื่น ๆ แม้ว่าผู้ใช้ทั้งหมดนั้นกำลังทำงานกับข้อมูลเดียวกันในฐานข้อมูลก็ตาม โดยสามารถระบุระดับของ isolation ที่ต้องการได้

การระบุระดับของ isolation สามารถควบคุมได้ว่าจะให้แต่ละทรานแซกชัน isolate จากกันมากน้อยเพียงใด ถ้าตั้งระดับของ isolation ที่สูง ก็มั่นใจได้ว่าแต่ละทรานแซกชันนั้น จะถูก isolate ออกจากทรานแซกชันอื่น ๆ ทั้งหมด แต่การตั้งระดับของ isolation สูงก็ทำให้ประสิทธิภาพในการทำงานที่ได้ลดลง เนื่องจากในการทำ isolation นั้นใช้วิธีการล็อกค่าในฐานข้อมูลเป็นหลัก แต่ถ้าตั้งระดับของ isolation ไว้ต่ำ ทรานแซกชันก็จะไม่ isolate แต่ประสิทธิภาพในการทำงานร่วมกันของหลาย ๆ ทรานแซกชันจะดีขึ้น

ดังนั้น ต้องพิจารณาให้รอบคอบว่า ต้องการความเป็น isolation สูงต่ำเพียงใดในการทำงาน การกำหนด transaction isolation มีอยู่ทั้งหมด 4 ระดับด้วยกัน ดังนี้

■ READ UNCOMMITTED

เป็นระดับการทำงานที่ต่ำที่สุด ไม่มีการรับประกันคุณสมบัติ isolation เลย แต่จะให้ประสิทธิภาพในการทำงานสูงสุด การทำงานในระดับนี้อาจเกิดปัญหา dirty read ขึ้นได้ โดยหากทรานแซกชันหนึ่งทำงานพร้อมกันไปกับทรานแซกชันอื่น แล้วอีกทรานแซกชันนั้นเขียนข้อมูลโดยยังไม่ได้ commit ทรานแซกชันเดิมจะอ่านค่านั้นเข้ามาได้ทำให้ได้ค่าที่ผิดพลาดไป โดยไม่ขึ้นกับระดับของ isolation ที่อีกทรานแซกชันหนึ่งใช้งานอยู่ นอกจากนี้ยังเกิดปัญหา unrepeatable read และ phantom phenomenon ได้อีกด้วย

■ READ COMMITTED

การทำงานในระดับนี้คล้ายกับระดับ READ UNCOMMITTED แตกต่างกันในระดับนี้โค้ดจะอ่านค่าข้อมูลที่ commit แล้วเท่านั้น จะไม่อ่านค่าที่ถูกเขียนลงไปยังฐานข้อมูลแล้วแต่ยังไม่ commit ดังนั้นระดับของ isolation ระดับนี้จึงสามารถแก้ปัญหา dirty read ได้ แต่ยังไม่สามารถแก้ปัญหาที่ซับซ้อน เช่นปัญหา unrepeatable read และ phantom phenomenon

■ REPEATABLE READ

เป็นการรับประกันที่สูงขึ้นไปจาก READ COMMITTED เมื่อใดก็ตามที่อ่านค่าข้อมูลที่ commit แล้วจากฐานข้อมูล สามารถจะอ่านค่าข้อมูลนั้นซ้ำได้และค่าที่ได้จะเป็นค่าที่เหมือนกับค่าที่เราอ่านขึ้นมาได้ครั้งแรก ซึ่งระดับของ isolation ที่ READ COMMITTED หรือต่ำกว่า ทรานแซกชันอื่นที่ทำงานอยู่พร้อมกันอาจจะ commit ข้อมูลในระหว่างการอ่านข้อมูลแต่ละครั้งได้ ทำให้ค่าของข้อมูลเดียวกันที่อ่านได้ในแต่ละครั้งไม่ตรงกัน การทำงานในโหมดนี้แก้ปัญหาที่เรียกว่า unrepeatable read

■ SERIALIZABLE

สามารถแก้ปัญหา phantom phenomenon คือการที่เซตของข้อมูลใหม่ถูกเพิ่มเข้ามาในฐานข้อมูลระหว่างการอ่านข้อมูลของทรานแซกชัน รวมถึงแก้ปัญหานั้น ๆ ที่กล่าวมาก่อนหน้านี้ทั้งหมด รับประกัน

ว่าทุก ๆ ทรานแซกชันจะทำงานเป็นลำดับกับทรานแซกชันอื่น ๆ และมีคุณสมบัติ ACID properties ของฐานข้อมูลครบถ้วน หมายความว่าแต่ละทรานแซกชันจะทำงานแยกกับทรานแซกชันอื่น ๆ อย่างแท้จริง

ตารางที่ 4-1 จะแสดงถึงปัญหาที่อาจเกิดในแต่ละระดับของ isolation

ISOLATION LEVEL	DIRTY READ?	UNREPEATABLE READS?	PHANTOM READ?
READ UNCOMMITTED	Yes	Yes	Yes
READ COMMITTED	No	Yes	Yes
REPEATABLE READ	No	No	Yes
SERIALIZABLE	No	No	No

ตารางที่ 4-1 ระดับของ Isolation

การกำหนด isolation ใน EJB ในกรณีของ bean-managed persistent จะกำหนด isolation โดยใช้ resource manager API เช่น JDBC โดยการเรียก `java.sql.Connection.SetTransactionIsolation(...)` ส่วนในกรณีของ container-managed persistent จะไม่สามารถกำหนดระดับของ isolation ใน deployment descriptor ได้ การกำหนดระดับของ isolation จะขึ้นอยู่กับผลิตภัณฑ์ที่ใช้เป็นคอนเทนเนอร์หรือฐานข้อมูล

4.5.5 ทรานแซกชันแบบกระจาย

นอกจากทรานแซกชันพื้นฐาน คือแอปพลิเคชันเซิร์ฟเวอร์หนึ่งตัวเชื่อมต่อกับฐานข้อมูลหนึ่งตัวแล้ว ยังสามารถที่จะทำ distributed transaction ได้เช่นเดียวกัน โดยขึ้นอยู่กับการรองรับของแอปพลิเคชันเซิร์ฟเวอร์ ซึ่ง distributed transaction ยึดหลักการเดียวกับทรานแซกชันธรรมดา โดยที่หากมีคอมพิวเตอร์เครื่องหนึ่งบนเครื่องคอมพิวเตอร์เครื่องหนึ่ง abort ทรานแซกชัน ทรานแซกชันทั้งหมดจะต้องถูก abort ด้วยเช่นกัน distributed transaction สามารถทำงานได้กับหลายแอปพลิเคชันเซิร์ฟเวอร์และหลายฐานข้อมูล หมายความว่าคอมพิวเตอร์ที่ติดตั้งในแอปพลิเคชันเซิร์ฟเวอร์หนึ่ง ทำงานในทรานแซกชันเดียวกันร่วมกับอีกคอมพิวเตอร์ซึ่งถูกติดตั้งในอีกแอปพลิเคชันเซิร์ฟเวอร์ ในการทำงานบิซิเนสเมธอดหนึ่ง ๆ เครื่องหลายเครื่องอาจจำเป็นต้องทำงานร่วมกัน distributed transaction จึงอนุญาตให้หลายแอปพลิเคชันเซิร์ฟเวอร์ซึ่งอาจเป็นซอฟต์แวร์จากผู้พัฒนาคนละรายกันทำงานร่วมกันในทรานแซกชันได้

4.5.6 ทรานแซกชันใน .NET

ทรานแซกชัน มีทั้งแบบโลคอล ซึ่งเป็นทรานแซกชัน ซึ่งมีขอบเขตการจัดการภายใต้แหล่งข้อมูลตัวเดียวกัน (single- transaction-aware data resource) หรือเรียกว่า internal transaction และอีกแบบ เป็นทรานแซกชันแบบกระจาย (Distributed transaction) เป็นการจัดการทรานแซกชันจากหลากหลายแหล่งข้อมูลร่วมกัน เช่น SQL Server , MSMQ

4.5.6.1 การจัดการทรานแซกชันแบบจัดการด้วยตนเอง (Manual Transaction)

Manual transaction ให้เราสามารถควบคุมขอบเขตการทำ transaction ได้อย่างชัดเจน (explicit) แต่ผลเสียก็คือ เราต้องจัดการขอบเขตของ transaction ให้ดีและ ที่สำคัญคือ เราต้องจัดการควบคุม distributed transaction เอง จึงมีงานมากมายให้เราทำ เราต้องควบคุมทุกๆ connection และ resource เพื่อให้เป็นไปตาม ACID ของ transaction ตัวอย่างของ transaction หนึ่งคือ ADO.NET สามารถกำหนด จุดเริ่มต้น, commit, rollback ของ transaction ได้เอง

4.5.6.2 การจัดการทรานแซกชันแบบการจัดการอัตโนมัติ (Automatic Transaction)

.Net มี MTS/COM+ ในการจัดการทรานแซกชันแบบการจัดการอัตโนมัติ ซึ่ง COM+ ใช้ Microsoft Distributed Transaction Coordinator (DTC) เป็นตัวจัดการทรานแซกชันทำให้ .NET แอปพลิเคชัน สามารถจัดการการใช้ทรัพยากรจาก ที่ต่างๆร่วมกันได้ เช่นจาก SQL SERVER , MSMQ , ORACLE ประสิทธิภาพของมันก็ขึ้นอยู่กับ DTC

ข้อดีคือ ง่ายในการเขียนโปรแกรม ไม่ต้องมากำหนดเองตรงๆ แต่ข้อเสียคือ ยากในการดูแลจัดการและเรามักจะละเลยไม่สังเกตถึงว่ามีการทำทรานแซกชันทั้งหมดฟังก์ชัน และในแบบแรกนั้น เราสามารถที่จะจัดการเกี่ยวกับ exception ที่ได้รับได้โดยตรง

ในแต่ละวิธีการจัดการทำทรานแซกชัน นั้นเป็นการแลกเปลี่ยนระหว่างประสิทธิภาพของแอปพลิเคชัน และ ความสามารถในการดูแลแอปพลิเคชัน

โดยสรุปรูปแบบการใช้งานทรานแซกชัน ใช้ Database Transaction ดูแลในเรื่องการจัดการทรานแซกชันใน สโตร์โปรซีเจอร์ (Stored procedure) Manual Transaction โดยการใช้ ADO.NET จะง่ายในการเขียน โค้ด และมีความยืดหยุ่นในการใช้ควบคุมทรานแซกชัน โดยทำได้โดยใช้คำสั่งภายนอกกำหนด begin transaction และ end transaction ส่วน Automatic Transaction จะมีประโยชน์มากกับงานประเภทที่มีการทำงานเกี่ยวกับทรานแซกชันจากหลายๆ แหล่งข้อมูลร่วมกัน

4.5.7 การจัดการทรานแซกชันใน เว็บเซอร์วิส

มีการจัดการทรานแซกชันในเว็บเซอร์วิสอยู่สองส่วนที่เราต้องจัดการ อันแรก คือ เราต้องการที่จะควบคุมทรานแซกชันเป็นส่วนหนึ่งของเว็บเซอร์วิสที่เราให้บริการ เนื่องจากเรามีการควบคุมระดับเน็ต

เวิร์กเลเยอร์ภายใน ASP.NET แอปพลิเคชัน เราสามารถใช้ COM+ เข้ามาช่วยได้ ส่วนที่สอง เรามองไปยังการรวมไปถึงทรานแซกชันของแอปพลิเคชันที่เรียกใช้ เว็บเซอร์วิส นั้น ซึ่งไม่มีการควบคุมในระดับของเน็ตเวิร์กเลเยอร์ ทำให้เป็นปัญหาในเรื่องของการควบคุมทรานแซกชันแบบกระจายขึ้น(Distributed Transaction Coordinator)

การเรียกใช้งานในเว็บเซอร์วิส นั้น ต้องทำการกำหนดค่าเว็บเมธอดแอททริบิวต์

[WebMethod(TransactionOption=TransactionOption.RequiresNew)]

คลาสนั้นต้องทำการ inherit มาจาก System.EnterpriseServices.ServicedComponent เป็นการตั้งให้ทำงานใน COM+ แล้ว COM+ จะทำการติดต่อกับ DTC ในการสร้าง distributed transaction และ จัดการ resource ทั้งหมดที่ติดต่อ โดยมีการกำหนดตัวเลือก ไว้ 5 แบบคือ

1. Disabled - อ็อบเจกต์ที่สร้างขึ้นจะไม่มีทรานแซกชัน ใน COM+ ทรานแซกชัน แต่สามารถติดต่อกับ DTC ในกรณีที่ต้องการการสนับสนุน
2. NotSupported - ไม่มีการทำทรานแซกชันเลย
3. Supported - ถ้าอ็อบเจกต์นั้นจะถูกรันในทรานแซกชัน context ของตัวที่สร้างมันขึ้นมาซึ่งมีการทำทรานแซกชัน , แต่ถ้าตัวมันเอง เป็นตัวเริ่ม(root - ไม่ได้ถูกใครสร้างมาอีกที) หรือ context ที่สร้างมัน ไม่ทำทรานแซกชัน มันก็จะไม่ทำทรานแซกชัน
4. Required - ถ้าอ็อบเจกต์นั้นจะถูกรันในทรานแซกชัน context ของตัวที่สร้างมันขึ้นมา มีการทำทรานแซกชัน , แต่ถ้าตัวมันเอง เป็นตัวเริ่ม(root - ไม่ได้ถูกใครสร้างมาอีกที) หรือ context ที่สร้างมัน ไม่ทำทรานแซกชัน มันก็จะทำทรานแซกชันใหม่
5. RequiresNew - บอกว่าอ็อบเจกต์ ต้องทำทรานแซกชัน ใหม่

ในการบ่งบอกถึง การ commit และ rollback นั้น เราทำได้โดยการ try /catch เพื่อตรวจจับ exception แต่เราก็สามารถให้มันทำการจัดการ commit ,abort ให้เองได้ด้วยการเรียกใช้

System.EnterpriseServices.AutoComplete

การกำหนดค่านี้ จะเป็นการกำหนดให้ไปเรียกใช้เซอร์วิสของคอมพลัส โดยอัตโนมัติ โดยมีโหมดให้เรียกใช้ TransactionOption เหมือนกับ การที่เขียนคอมโพเนนต์ที่ สืบทอดมาจาก EnterpriseServices ถ้ามี exception ขึ้นทรานแซกชันจะทำการ roll back โดยอัตโนมัติ แต่ถ้าการทำงานนั้นอยู่ภายใต้ try...catch ทรานแซกชันจะคอมมิท ยกเว้นแต่เราจะทำการ SetAbort ด้วยตัวเอง โดยเรียกใช้ผ่านอ็อบเจกต์ ContextUtil

4.5.7.1 ตัวอย่างการใช้งาน เว็บเซอร์วิสที่มีการใช้ทรานแซกชัน

มี 2 ฟังก์ชันคือ UpdateDoctor ทำการอัปเดตสถานะของหมอ และ InsertLog เพื่อทำการเก็บบันทึกการเปลี่ยนแปลง และมีฟังก์ชันที่เป็นเว็บเมธอด ให้ใช้งานซึ่งจะไปเรียก 2 ฟังก์ชันดังกล่าว

โดยหลังจากทำการอัปเดตสถานะแล้วก็จะบันทึกการเปลี่ยนแปลงเก็บไว้ด้วย แต่จะมีการโยน exception ออกมาทำให้เกิดการ roll back ผลลัพธ์ที่ได้นั้นจะส่งผลให้ ฟังก์ชัน UpdateDoctor เกิดการ roll back ด้วยเช่นกัน

```
[WebMethod(TransactionOption=TransactionOption.RequiresNew)]
public void UpdateStatus(int DoctorID, int StatusID)
{
    UpdateDoctor(DoctorID, StatusID);
    InsertLog(DoctorID, StatusID);
}
```

4.5.7.2 การเรียกใช้เว็บเซอร์วิสที่มีทรานแซกชัน

จากส่วนแรก การเรียกใช้งานก็เป็นไปตามปกติเหมือนการเรียกใช้เว็บเซอร์วิสโดยทั่วไป ในหัวข้อนี้จะพูดถึงการเรียกใช้เว็บเซอร์วิสให้เป็นส่วนหนึ่งของทรานแซกชันภายใต้คอนเท็กซ์ของไคลเอนต์แอปพลิเคชัน จากเอกสารของไมโครซอฟท์กล่าวว่า

“เว็บเซอร์วิสเมธอดแต่ละอันจะเป็นส่วนหนึ่งของทรานแซกชันของตัวเองเท่านั้น เนื่องจาก เว็บเซอร์วิสเมธอด สามารถเป็นได้เพียง root ของทรานแซกชัน เท่านั้น “

หมายความว่า เราไม่สามารถที่จะสร้างแอปพลิเคชันที่เรียกใช้เว็บเซอร์วิสในทรานแซกชันคอนเท็กซ์ในโลคอลเมธอดได้

ปัญหาที่เกิดขึ้นก็คือ สมมุติว่า เรามี ASP.NET แอปพลิเคชัน มีการเรียกใช้เว็บเซอร์วิสเมธอดสองตัวที่เป็นทรานแซกชันเมธอด โดยที่ ASP.NET เรามีการกำหนดให้เป็นทรานแซกชันคอนเท็กซ์ด้วย ถ้าเว็บเมธอดอันที่สองเกิดการ roll back เมธอดแรกจะไม่ roll back ด้วย

ถึงแม้เราจะการกำหนดให้เป็นทรานแซกชันทั้งระดับของโลคอลใน ASP.NET และระดับเว็บเซอร์วิสก็ตามแต่ก็ไม่สามารถแก้ปัญหานี้ได้เนื่องจาก เว็บเซอร์วิสไม่ได้เป็นส่วนหนึ่งของทรานแซกชันที่โลคอลแอปพลิเคชัน

4.5.7.3 ปัญหาในการเรียกใช้ทรานแซกชันเว็บเซอร์วิส

โดยตัวนิยามแล้ว เว็บเซอร์วิสแล้วไม่ตรงตามมาตรฐานของ ACID มันเป็น loosely coupled มันจึงไม่สามารถควบคุมได้ จากแต่เดิมใน DTC (distributed transaction control) ใช้ข้อมูลของเน็ตเวิร์กเลเยอร์ในการควบคุมความถูกต้องของทรานแซกชัน เนื่องจากทุกๆเน็ตเวิร์กทราฟฟิกของไมโครซอฟท์ ทราน

แชกชันแบบกระจาย จะรายงานความถูกต้องผ่านเน็ตเวิร์กไปยังผู้เรียก โดยตัว DTC จะใช้ข้อมูลนี้ในการพิจารณาตัดสินใจ roll back ในกรณีที่ต้องทำ ถ้าเกิดมีการล้มเหลวระดับเน็ตเวิร์ก ตัว DTC จะรับรู้แล้วจัดการตามสิ่งที่เกิดขึ้น

เรามี WSDL ในการอธิบาย อินเทอร์เฟซของรีโมทเซอร์วิส ซึ่งไม่ได้อธิบายเกี่ยวกับการเรียกใช้งานกับ DTC ซึ่งก็เป็นความจริงที่ว่า คอมพลัสไม่ได้จัดการกับทั้งหมดของโลก .NET และไม่ได้เกี่ยวข้องกับ WSDL

จากเหตุผลที่กล่าวมาข้างต้น เราจึงไม่สามารถที่จะรวมเว็บเซอร์วิสเข้ากับ ทรานแซกชันใน ASP.NET ได้ นี่เป็นอีกจุดอ่อนหนึ่งของเว็บเซอร์วิสโดยทั่วไป ไม่ได้เฉพาะการอิมพลีเมนต์เว็บเซอร์วิสของไมโครซอฟท์เท่านั้น

4.5.7.4 แนวทางการแก้ไขปัญหา

ในการพัฒนาแอปพลิเคชันที่มีการทำงานของทรานแซกชันด้วยเว็บเซอร์วิส ระบบจำเป็นต้องมี Transaction Monitor ซึ่งทำหน้าที่ในการดูแลและจัดการทรานแซกชันทั้งหมด (เช่น roll back ในกรณีที่มีปัญหาเกิดขึ้นกับทรานแซกชัน) แน่นอนว่า TM จะต้องทำงานร่วมกันกับแอปพลิเคชันอย่างใกล้ชิด เพื่อให้มีคุณสมบัติพิเศษ เช่น การล็อกข้อมูลที่ใช้งานร่วมกัน หรือการย้อนกลับเมธอดที่ทำงานไปแล้ว ในกรณีของการ roll back เป็นต้น ดังนั้น TM จึงจำเป็นต้องมีการทำงานกับระบบเว็บเซอร์วิสอย่างใกล้ชิดซึ่งส่งผลให้ความอิสระต่อกันในเชิงโครงสร้างของซอฟต์แวร์สูญหายไป

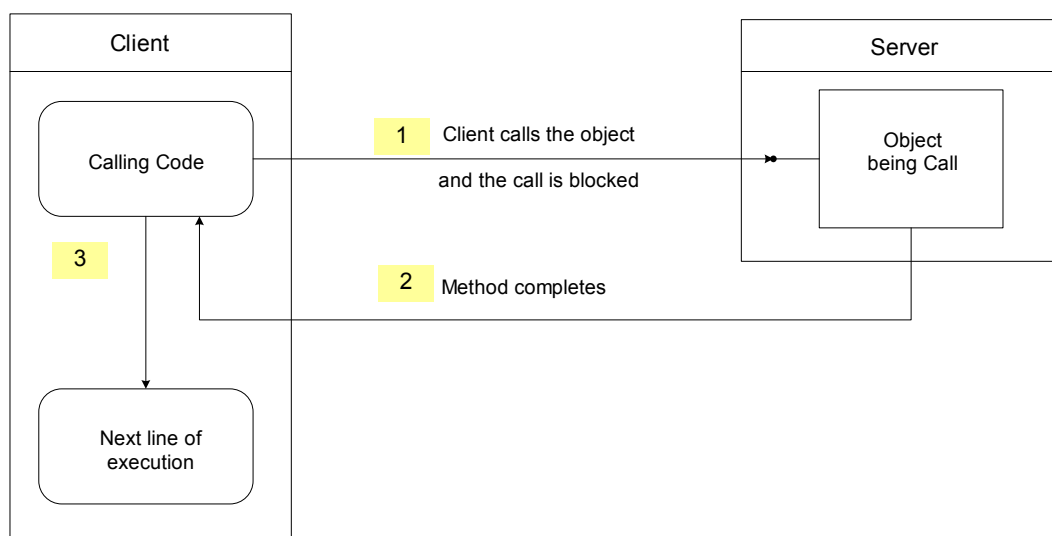
มีโซลูชันในการแก้ปัญหาที่ถูกนำเสนอออกมาใช้กับระบบงานแบบทรานแซกชัน ซึ่งผู้ที่เกี่ยวข้องกำลังพัฒนามาตรฐานสำหรับเว็บเซอร์วิส เพื่อให้รองรับระบบการจัดการทรานแซกชัน ตัวอย่างของมาตรฐานเหล่านี้ได้แก่ Transaction Authority Markup Language(XAML) และ Business Transaction Protocol (BTP) ซึ่งคาดว่าจะได้รับการแก้ไขโดยสมบูรณ์ราวปี 2547

อีกวิธีการแก้ปัญหานี้ก็คือ สำหรับระบบที่มีการใช้งานภายในองค์กร ที่มีการใช้งาน .NET เพียงระบบเดียว เราก็สามารถเลือกใช้โปรโตคอลการสื่อสาร .NET Remoting ได้ ซึ่งถูกพัฒนามาจาก DCOM ซึ่งสามารถจัดการปัญหาดังกล่าวได้

4.6 การเรียกใช้เว็บเซอร์วิสแบบอซิงโครนัส(Asynchronous)

4.6.1 การประมวลผลแบบซิงโครนัส(Synchronous Processing)

การทำงานแบบซิงโครนัสนั้น ประกอบไปด้วย ฟังก์ชันและคอมโพเนนต์ ที่มีการเรียกใช้งาน การเรียกใช้งานจะถูกบล็อกจนกว่าการทำงานของฟังก์ชันนั้นจะครบถ้วนเรียบร้อย รูปแบบการทำงานเป็นไปตามรูปที่ 4-6



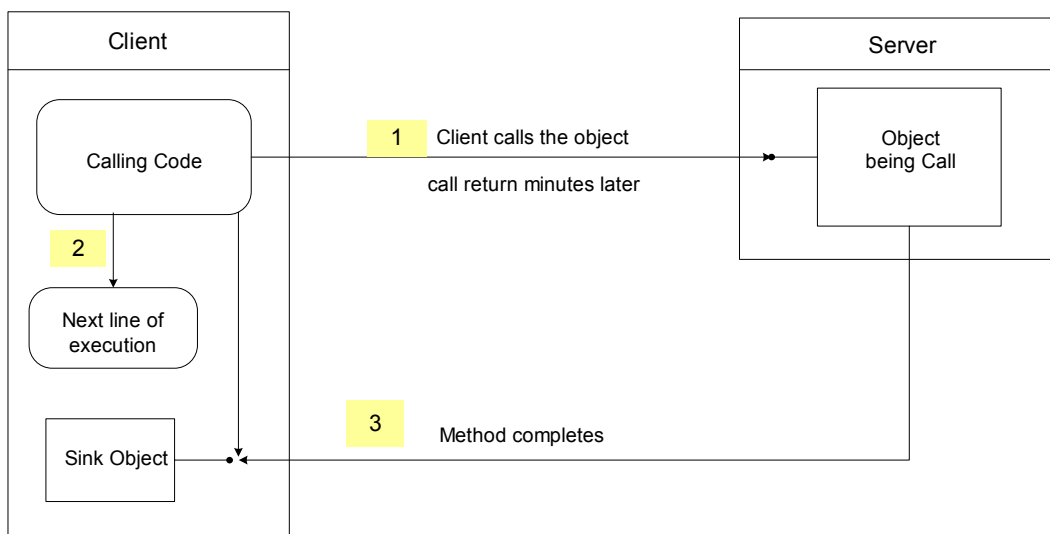
รูปที่ 4-6 การติดต่อแบบซิงโครนัส

- 1.ไคลเอนต์ทำการเรียกใช้ ฟังก์ชันการทำงานไปยังอ็อบเจกต์คอมโพเนนต์ การทำงานฝั่งไคลเอนต์ถูกบล็อก
- 2.เมื่อฝั่งเซิร์ฟเวอร์ทำงานเสร็จส่งผลลัพธ์กลับมา
- 3.เมื่อได้รับผลลัพธ์ ไคลเอนต์ ทำการประมวลผลต่อได้

การเรียกใช้งานเว็บเซอร์วิส แบบซิงโครนัสเป็นการเรียกใช้งานตามแบบปกติทั่วไป ซึ่งง่ายในการจัดการ แต่มีข้อเสียคือเรื่องของประสิทธิภาพโดยฝั่งไคลเอนต์ต้องคอยรอผลลัพธ์จากเว็บเซอร์วิส

4.6.2 การประมวลผลแบบอซิงโครนัส(Asynchronous Processing)

การเรียกใช้งานแบบอซิงโครนัส จะไม่มีการบล็อกจากคอมโพเนนต์ฝั่งเซิร์ฟเวอร์ ไคลเอนต์จะรู้ว่าการประมวลผลเสร็จเรียบร้อยแล้วจาก polling หรือจาก การอินเทอร์รัพจากโปรแกรม โมเดลการทำงานแบบนี้ดังรูปที่ 4-7



รูปที่ 4-7 การติดต่อแบบอซิงโครนัส

- 1.ไคลเอนต์ทำการเรียกไปยังเซิร์ฟเวอร์
- 2.ไคลเอนต์สามารถที่จะประมวลผลโค้ดในบรรทัดต่อไปได้เลย โดยไม่ต้องรอผลลัพธ์
- 3.เมื่อฝั่งเซิร์ฟเวอร์ทำงานเสร็จจะส่งผลลัพธ์กลับมาให้ ยังจุดที่สร้างขึ้นเพื่อรอรับผล(Sink point)

การเลือกโมเดลในการพัฒนาระบบ ไม่ได้เป็นเพียงเรื่องเล็กน้อย เราต้องทำการตัดสินใจจากความต้องการของระบบที่เราพัฒนาอย่างรอบคอบเพื่อตัดสินใจใช้งานการเรียกใช้แบบอซิงโครนัสหรือไม่ โดยอาจจะนำคำแนะนำต่อไปนี้เพื่อมาช่วยในการตัดสินใจ

- การเรียกใช้งานแบบอซิงโครนัส จะช่วยขยายการรองรับการใช้งานของระบบ(scalability) ถ้าไคลเอนต์เป็น ASP.NET แอปพลิเคชัน หรือเป็น เว็บเซอร์วิสก็ตาม เนื่องจากการเรียกใช้แบบซิงโครนัสจะมีการบล็อกทำให้เทรคของ ASP.NET ที่ทำงานเกิดการรอ(stall) ส่งผลให้การร้องขออื่นๆมายังแอปพลิเคชัน ต้องเข้าคิว ส่งผลกระทบต่อประสิทธิภาพของระบบ การเรียกใช้แบบอซิงโครนัสแทนอย่างน้อยก็สามารถที่จะปลดปล่อยเทรค ASP.NET ที่ไม่ได้เรียกใช้เว็บเซอร์วิสในขณะนั้น
- การเรียกใช้งานแบบบริโมทโพรซีเยอร์คอลไปยังเว็บเซอร์วิสนั้นอาจจะใช้เวลานานก่อนการส่งผลลัพธ์กลับ ในกรณีนี้การเรียกใช้แบบอซิงโครนัสจะมีประโยชน์ โดยฝั่งไคลเอนต์สามารถที่จะทำงานอย่างอื่นได้ ถ้าไม่จำเป็นต้องใช้ผลลัพธ์ในทันที
- ในกรณีที่ไคลเอนต์ต้องการเรียกใช้งานไปยังหลายเว็บเซอร์วิสพร้อมๆกันหลายๆที่ หรือที่เดียวกันแต่หลายฟังก์ชัน การใช้งานแบบอซิงโครนัสจะเป็นประโยชน์มากเนื่องจากไคลเอนต์สามารถที่จะเรียกใช้งานไปยังทุกๆที่พร้อมๆกันได้เลย ไม่ต้องคอยรอผลลัพธ์ของแต่ละที่แล้วค่อยเรียกต่อแบบในกรณีของซิงโครนัส ทำให้เพิ่มประสิทธิภาพได้

4.6.3 การเรียกใช้แบบอซิงโครนัสใน .NET

.NET มีโมเดลการเรียกใช้งานแบบอซิงโครนัส มาให้พร้อมอยู่แล้วดังนี้

- .NET เฟรมเวิร์ก จะให้บริการเซอร์วิสที่จำเป็นในการรองรับโมเดลการใช้งานแบบอซิงโครนัส
- การเรียกใช้ที่อยู่โค้ดไคลเอนต์ เรียกใช้งานในฟังก์ชันที่จำเป็นต้องใช้
- ไม่จำเป็นที่อ็อบเจกต์ที่ถูกเรียกต้องทำการเขียนโค้ดเพิ่มเติมเพื่อรองรับการใช้งาน ตัว CLR จะทำการจัดการให้
- รันไทม์ CLR ให้บริการตรวจสอบเกี่ยวกับความปลอดภัยในเรื่องของชนิดตัวแปร โดยการเรียกใช้ ดีลิกเกต(delegate) ซึ่งก็เหมือนกับ พอยน์เตอร์ไปยังฟังก์ชันที่มีส่วนของการอธิบายตัวเองด้วยซึ่งต่างจากแต่ก่อนใน C++ พอยน์เตอร์ไปยังฟังก์ชันจะไม่บอกจำนวนพารามิเตอร์ที่ส่งผ่าน ชนิด ของตัวแปรพารามิเตอร์ และพอยน์เตอร์ฟังก์ชัน ก็เหมือนพอยน์เตอร์ทั่วไป (เป็น void)

4.6.4 การเรียกใช้งานเว็บเซอร์วิสแบบอซิงโครนัส

การเรียกใช้เว็บเซอร์วิสแบบอซิงโครนัส ทำให้เราสามารถไปทำงานอย่างอื่นได้โดยไม่ถูก block เพื่อที่จะรอผลของ services นั้นๆ มีประโยชน์ในกรณีที่ services นั้นมีการเวลาประมวลผลนาน หรือในกรณีที่เรารเรียกใช้ services จากหลายที่เราส่งผลไปพร้อมกันทั้ง 3 ที่ โดยไม่ต้องมานั่งรอผลทีละที

การทำงาน จะเรียกใช้ฟังก์ชันโดยเติม Begin นำหน้า เช่นฟังก์ชัน Hello() จะเป็น BeginHello() โดยที่ฝั่งเว็บเซอร์วิสเราไม่ต้องเขียนโค้ดเพื่อรองรับเลย การเรียกใช้กำหนดตามความต้องการทางไคลเอนต์ เราสามารถจบการทำงาน asynchronous ได้หลายวิธี

1. ทำการจบโดยเรียกใช้ EndHello() ซึ่งจะจบก็ต่อเมื่อ services นั้นทำงานเสร็จ ไม่งั้นก็จะถูกบล็อกรอเช่นกัน
2. ตรวจสอบจาก flag IAsyncResult.IsComplete ซึ่งจะมีค่าเป็น true เมื่อทำงานเสร็จสิ้น

ตัวอย่างการใช้งาน

```
IAsyncResult AsyncResult; //ประกาศตัวแปรแบบเพื่อรอรับผลของการเรียกใช้
AsyncResult = ObjectWS.BeginHello(); //เรียกใช้งานแบบอซิงโครนัส
while(AsyncResult.IsCompleted == false)
{
    //other job here
    // วนลูปทำการเช็คจากแฟล็กว่าเสร็จรึยัง
}
```

3. ในขณะที่เรียก BeginHello() ทำการใส่พารามิเตอร์ ดิเลเกต(delegate) ที่ต้องการให้ทำงาน เมื่อเว็บเซิร์ฟเวอร์ทำงานเสร็จจะเรียกกลับมายังฟังก์ชันหรือคิเลเกตนั่นๆ

ขั้นตอนการใช้งาน

- ทำการประกาศ call back delegate โดยส่งค่าฟังก์ชันที่ต้องการให้เรียกกลับไปด้วย

```
AsyncCallback AsyncCallback = new AsyncCallback(MyCallBack);
```

- ฟังก์ชันที่จะทำการเรียกใช้ ต้องรับค่าพารามิเตอร์ IAsyncResult ด้วย และภายในฟังก์ชันต้องมีการทำการเรียก END เพื่อจบการใช้งานฟังก์ชันที่เรียกไปยังเว็บเซิร์ฟเวอร์ด้วย

```
private void MyCallBack(System.IAsyncResult AsyncResult
```

- การเรียกใช้งาน Begin Hello() ต้องมีการส่งผ่านค่าพารามิเตอร์ดังนี้

```
AsyncResult = objWS.BeginHello(parameter1,AsyncCallback,asyncState);
```

Parameter1 เป็นพารามิเตอร์ที่ฟังก์ชัน Hello ต้องการอาจมีหลายพารามิเตอร์ก็ได้

AsyncCallback ค่าของ delegate ที่ต้องการให้มีการเรียกกลับเมื่อทำงานเสร็จ

AsyncState ใช้ในการส่งข้อมูลเกี่ยวกับข้อมูลของการเรียกแบบอซิงโครนัสไปยัง delegate

สำหรับตัวแปรสองตัวทำขึ้นนั้น ถ้าไม่ต้องการใช้งานก็ทำการส่งค่า null ไปแทน

4.6.5 ปัญหาที่เกิดขึ้นกับการนำไปใช้งานจริง

จากการทำการทดสอบการใช้งานใน 3 รูปแบบข้างต้นสามารถทำได้ตามทฤษฎีในโคลเอนต์วินโดวส์ฟอร์ม แต่สำหรับโคลเอนต์ที่เป็น ASP.NET แอปพลิเคชัน ผลลัพธ์ที่ได้จากการทดลองผิดพลาดดังนี้

ผลการทดลองในรูปแบบของASP.NET เว็บฟอร์ม

1. แบบ END ฟังก์ชัน : ได้ผลลัพธ์ถูกต้อง
2. ทำการเช็คผลจากแฟล็ก AsyncResult.IsCompleted : รอผลที่ได้เวลานานมาก จนสุดท้ายไม่มีผลขึ้นแสดง
3. ทำการเรียกกลับมายัง delegate : ไม่สามารถ เรียกไปยังฟังก์ชัน delegate ที่ต้องการได้ ผลที่ได้คือหน้าจอเดิมที่ทำการเรียก

จากการเข้าไปค้นหาข้อมูลจากอินเทอร์เน็ตได้ข้อมูลดังนี้ สาเหตุที่ไม่สามารถทำงานในเว็บฟอร์มได้เนื่องจาก ตัวโคลเอนต์ซึ่งในที่นี้คือ เว็บเซิร์ฟเวอร์จะไม่ส่งผลลัพธ์กลับมาจนกว่า การเรียกใช้งานทั้งหมดจะเสร็จเรียบร้อย หมายถึงขั้นตอนตั้งแต่เว็บแอปพลิเคชันถูกเรียกใช้งาน,เรียกไปยังเว็บเซิร์ฟเวอร์ จนกระทั่งเว็บส่งผลลัพธ์กลับมา วิธีที่เดียวจะเรียกใช้แบบอซิงโครนัสได้คือ เรียกใช้โดยตรงจากโคลเอนต์ที่ไม่ผ่านเว็บเซิร์ฟเวอร์ ซึ่งสามารถทำได้โดยโคลเอนต์สคริปต์ เช่น java script ซึ่งต้องมีการกำหนด time out ให้นานพอสมควร

การเรียกใช้งานแบบอซิงโครนัสโดยไคลเอนต์เป็นจาวานั้นไม่สามารถทำได้เนื่องจากตัวจัดการการเรียกใช้แบบนี้ ฝังอยู่ในอินฟราสตรักเจอร์ของ CLR ใน .NET เราจึงไม่สามารถเรียกใช้ความสามารถนี้ในจาวาได้

เนื่องจากมันไม่สามารถเรียกใช้จากค่ายาจาได้ รวมทั้งค่อนข้างยุ่งยากในการเรียกใช้งานกรณีไคลเอนต์เป็นเว็บฟอร์ม ดังนั้นจึงไม่เหมาะในการนำมาพัฒนา เราจะทำการศึกษาการทำงานแบบอซิงโครนัส โดยใช้เทคโนโลยีของคิว(MSMQ) ต่อไป

การแก้ปัญหาอ้างอิงจาก

www.dotnet247.com/247reference/messages/21/106614.aspx

www.dotnet247.com/247reference/messages/24/123480.aspx